❶ Set to `None` to disable ARD, `2` to enable ARD.

5. We intialize our GP and train it using gradient descent for 500 iterations as follows.

```
# declare the GP
noise = 1e-4

likelihood = gpytorch.likelihoods.GaussianLikelihood()
model = GPModel(train_x, train_y, likelihood)

# fix the hyperparameters
model.likelihood.noise = noise

# train the hyperparameters
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
mll = gpytorch.mlls.ExactMarginalLogLikelihood(likelihood, model)

model.train() ❶
likelihood.train() ❶

losses = []
for i in tqdm(range(500)): ❷
    optimizer.zero_grad() ❷

    output = model(train_x) ❷
    loss = -mll(output, train_y) ❷

    loss.backward() ❷
    losses.append(loss.item()) ❷

    optimizer.step() ❷

model.eval() ❸
likelihood.eval() ❸
```

❶ Enable training model.

❷ Gradient descent to optimize the GP's hyperparameters.

❸ Enable prediction model.

After these 500 iterations, we inspect the length scale by printing out the following value.

```
>>> model.covar_module.base_kernel.lengthscale
tensor([[1.1535]])
```

In other words, the optimized length scale is equal to roughly 1.15.

6. Setting `ard_num_dims=2` in the `GPModel` class and rerunning all the code cells, we obtain the following values for the length scales.

```
>>> model.covar_module.base_kernel.lengthscale
tensor([[1.6960, 0.8739]])
```

Here, the length scale of the first dimension is large (roughly 1.70), while the length scale of the second dimension is small (roughly 0.87). This corresponds to the fact that the objective function varies more along the second dimension.

©Manning Publications Co. To comment go to liveBook
https://livebook.manning.com/#!/book/bayesian-optimization-in-action/discussion

Licensed to ashwin sampath <ashwin.manning@gmail.com>